

Toronto Bicycle Thefts Data Analysis

DAMG 7370

Team

Chaitanya Sai Gandhi

Cloud Infra & ETL

EC2, Kafka, S3, Python ETL

Mudra Pandya

Data Modeling &
SQL

RDS schema, SQL queries & views

Qi Han

BI & Dashboards

QuickSight visuals & storytelling

The Problem

Urban Challenge & Hidden Patterns



Bicycle theft is a persistent urban challenge in Toronto



Bikes are **easy to steal**, resell, and move across neighborhoods



Individual thefts reported, but **bigger picture** remains hidden



Raw police data (CSV) isn't actionable for decision-makers

Key Strategic Questions



Where are bikes stolen most often? ?



When does theft risk peak? ?



Which bike types/values are most targeted? ?

Why This Dataset?

Real-world Relevance



(genuine urban problem,
helps actual stakeholders)

Rich Data Quality



(35K+ records,
coordinates, multiple
dimensions)

Perfect for Streaming



(event-level,
time-stamped,
open/free access)

Our Solution

Data Transformation & Source



Transform raw police data into interactive dashboards

- **Data:** Toronto Police Open Data (35,000+ theft records)

Empowering Target Users



Cyclists → understand personal risk



City planners → allocate prevention resources



Law enforcement → identify patterns & hotspots

AWS Services Used



EC2 (t3.xlarge)

Host Kafka, ZooKeeper,
Python ETL



Apache Kafka

Message streaming
layer



S3

Data lake for cleaned dataset



Amazon RDS

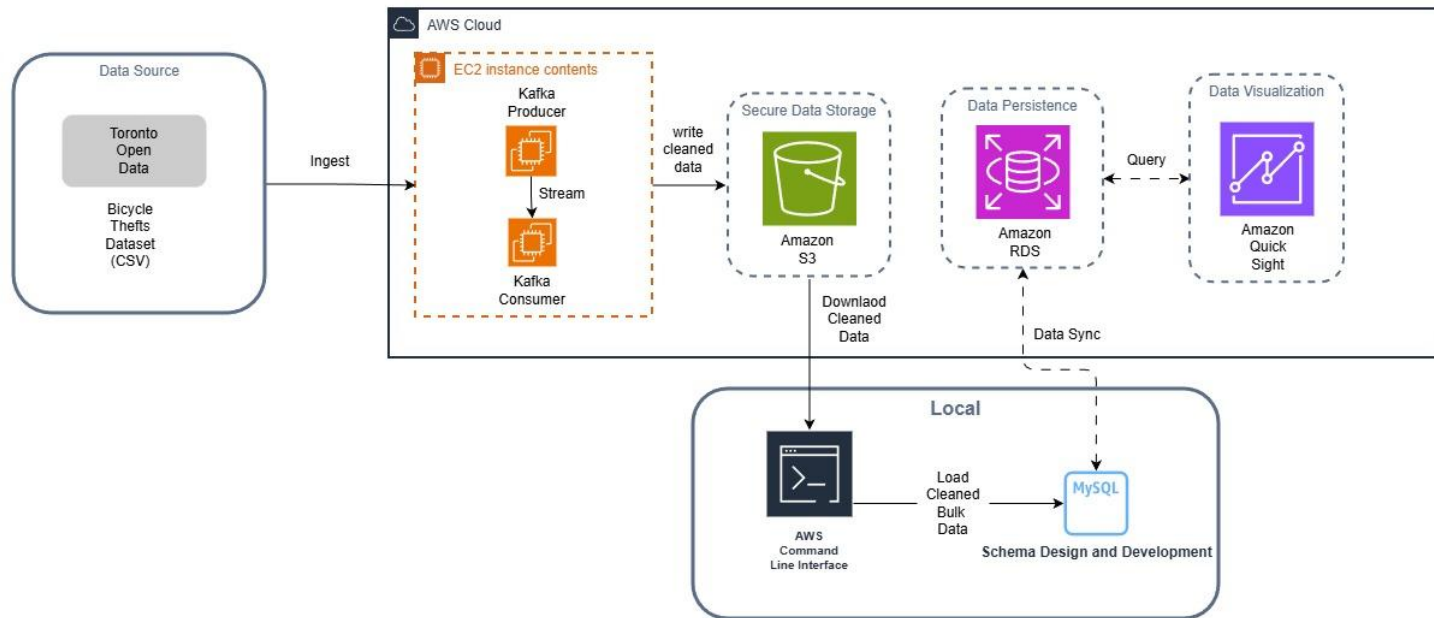
Relational Database Storage



QuickSight

Interactive dashboards

Architecture Overview



Why This Architecture?



EC2

Amazon EC2 (Compute & Control)

- Full root access for custom tuning (JVM, Kafka).
- Direct SSH debugging & config.
- Cost-effective single instance (t3.xlarge).



S3

Amazon S3 (Data Lake Storage)

- Cheap, durable object storage.
- Central repo for cleaned data.
- Easy collaboration & sharing.



RDS

Amazon RDS MySQL (Structured Data Layer)

- Relational schema & SQL queries.
- Create views for analysis.
- Fine-grained access control.



QuickSight

Amazon QuickSight (Visual Analytics)

- Native AWS integration (no code).
- Simplified networking & auth.
- Focus on interactive visuals & storytelling.

EC2 > Instances > i-06a7cb9effa159059

EC2

- Dashboard
- EC2 Global View
- Events
- Instances
 - Instances
 - Instance Types
 - Launch Templates
 - Spot Requests
 - Savings Plans
 - Reserved Instances
 - Dedicated Hosts
 - Capacity Reservations
 - Capacity Manager
- Images
 - AMIs
 - AMI Catalog
- Elastic Block Store
 - Volumes
 - Snapshots
 - Lifecycle Manager
- Network & Security
 - Security Groups
 - Elastic IPs
 - Placement Groups
 - Key Pairs
 - Network Interfaces

Instance summary for i-06a7cb9effa159059 (bicycle-theft-ec2-v2)

Updated less than a minute ago

[Connect](#) [Instance state](#) [Actions](#)

Instance ID i-06a7cb9effa159059	Public IPv4 address 107.20.24.237 open address	Private IPv4 addresses 172.31.24.229
IPv6 address -	Instance state Running	Public DNS ec2-107-20-24-237.compute-1.amazonaws.com open address
Hostname type IP name: ip-172-31-24-229.ec2.internal	Private IP DNS name (IPv4 only) ip-172-31-24-229.ec2.internal	Elastic IP addresses -
Answer private resource DNS name IPv4 (A)	Instance type t3.xlarge	AWS Compute Optimizer finding Opt-in to AWS Compute Optimizer for recommendations. Learn more
Auto-assigned IP address 107.20.24.237 [Public IP]	VPC ID vpc-0e63810c72fa82828	Auto Scaling Group name -
IAM Role EC2-BicycleTheft-S3Role	Subnet ID subnet-0a3898281a516ddb5	Managed false
IMDSv2 Required	Instance ARN arn:aws:ec2:us-east-1:418272788584:instance/i-06a7cb9effa159059	
Operator -		

Details

Instance details	Monitoring	Platform details
AMI ID ami-0f00d706c4a80fd93	disabled	Linux/UNIX
AMI name al2023-ami-2023.9.20251117.1-kernel-6.12-x86_64	Allowed image -	Termination protection Disabled
Stop protection Disabled	Launch time Fri Dec 05 2025 23:12:07 GMT-0500 (Eastern Standard Time) (2 days)	AMI location amazon/al2023-ami-2023.9.20251117.1-kernel-6.12-x86_64
Instance reboot migration Default (On)	Instance auto-recovery Default	Lifecycle normal

CloudShell Feedback Console Mobile App

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

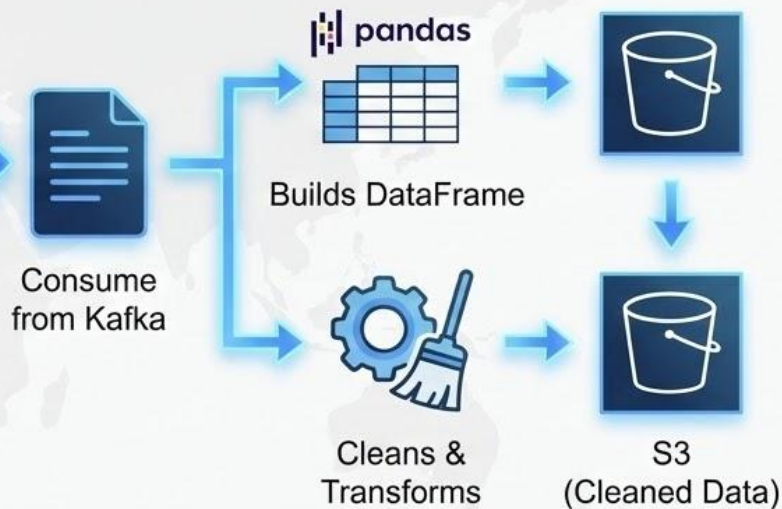
This screenshot shows the EC2 instance (bicycle-theft-ec2-v2) that hosts our data processing stack. It is a t3.xlarge instance in us-east-1 with public IP 107.20.24.237 and IAM role EC2-BicycleTheft-S3Role. On this machine we installed Java, Kafka, ZooKeeper and Python, and we run the Kafka producer and consumer ETL scripts that read the raw bicycle theft CSV, stream it through Kafka, clean it with pandas, and upload the cleaned dataset to S3.

ETL Pipeline

Producer Script:



Consumer Script:



Data Cleaning Steps



Standardize columns → lowercase, underscores



Parse dates → handle invalid/missing values safely



Convert numerics → coerce errors to NaN



Normalize text → uppercase, strip whitespace



Drop rows missing dates or coordinates



Remove duplicate records



Drop redundant geometry column

```
print(f"Reading raw CSV from: {INPUT_CSV}")
df = pd.read_csv(INPUT_CSV)

print("Raw DataFrame shape:", df.shape)
df.head()
```

[3]

... Reading raw CSV from: [D:\NEU\Fall 2025\DMG 7370\Bicycle-Theft\bicycle-thefts-4326.csv](#)
Raw DataFrame shape: (39473, 30)

...

	_id	EVENT_UNIQUE_ID	PRIMARY_OFFENCE	OCC_DATE	OCC_YEAR	OCC_MONTH	OCC_DOW	OCC_DAY	OCC_DOY	OCC_HOUR	...	BIKE_MAKE	BIKE_MODEL	BIKE_TYPE	BIKE_SPEED	BIKE_COLOUR	BIKE_COST	STATUS	LONG_WGS84	LAT_W
0	1	GO-20141263544	B&E	2013-12-26	2013	December	Thursday	26	360	19	...	FELT	F59	RC	21.0	SILRED	1300.0	STOLEN	-79.395643	43.64
1	2	GO-20141263784	PROPERTY - FOUND	2014-01-01	2014	January	Wednesday	1	1	18	...	TREK	SOHO S	RG	1.0	BLK	NaN	RECOVERED	-79.414654	43.64
2	3	GO-20141261431	THEFT UNDER	2014-01-01	2014	January	Wednesday	1	1	7	...	SUPERCYCLE	NaN	MT	10.0	NaN	NaN	STOLEN	-79.443645	43.64
3	4	GO-20149000090	THEFT UNDER	2014-01-01	2014	January	Wednesday	1	1	12	...	GI	TCX2 (2010)	OT	9.0	BLU	1019.0	STOLEN	-79.393760	43.64
4	5	GO-20141267465	THEFT UNDER	2013-09-30	2013	September	Monday	30	273	0	...	NORCO	CHARGER 9.2	MT	21.0	BLK	750.0	STOLEN	-79.404678	43.64

5 rows x 30 columns

Raw Toronto Bicycle Thefts dataset loaded in pandas

This cell shows the initial load of the `bicycle-thefts-4326.csv` file into a pandas DataFrame. The raw dataset has **39,473 rows and 30 columns**, including event details, dates, location context, bike attributes and geospatial fields. At this stage we can already see issues such as mixed data types and missing values in fields like `BIKE_TYPE` and `BIKE_COST`, which motivates the subsequent cleaning and standardization steps in our ETL pipeline.

```
df.columns

[4] Python

''' Index(['_id', 'EVENT_UNIQUE_ID', 'PRIMARY_OFFENCE', 'OCC_DATE', 'OCC_YEAR',
        'OCC_MONTH', 'OCC_DOW', 'OCC_DAY', 'OCC_DOY', 'OCC_HOUR', 'REPORT_DATE',
        'REPORT_YEAR', 'REPORT_MONTH', 'REPORT_DOW', 'REPORT_DAY', 'REPORT_DOY',
        'REPORT_HOUR', 'DIVISION', 'LOCATION_TYPE', 'PREMISES_TYPE',
        'BIKE_MAKE', 'BIKE_MODEL', 'BIKE_TYPE', 'BIKE_SPEED', 'BIKE_COLOUR',
        'BIKE_COST', 'STATUS', 'LONG_WGS84', 'LAT_WGS84', 'geometry'],
        dtype='object')

# Standardize: lowercase, strip spaces, replace spaces with "_"
df.columns = [str(c).strip().lower().replace(" ", "_") for c in df.columns]

print("Renamed columns:")
df.columns

[5] Python

''' Renamed columns:

''' Index(['_id', 'event_unique_id', 'primary_offence', 'occ_date', 'occ_year',
        'occ_month', 'occ_dow', 'occ_day', 'occ_doy', 'occ_hour', 'report_date',
        'report_year', 'report_month', 'report_dow', 'report_day', 'report_doy',
        'report_hour', 'division', 'location_type', 'premises_type',
        'bike_make', 'bike_model', 'bike_type', 'bike_speed', 'bike_colour',
        'bike_cost', 'status', 'long_wgs84', 'lat_wgs84', 'geometry'],
        dtype='object')
```

Standardizing column names for consistent downstream processing

In this step I inspect the original column names from the raw CSV and then standardize them by converting everything to lowercase, stripping extra spaces, and replacing spaces with underscores (e.g., **OCC_DATE** → **occ_date**, **REPORT_YEAR** → **report_year**). Having clean, consistent column names makes it much easier to write SQL, build views in MySQL, and reference fields later in the Kafka ETL code and QuickSight dashboards.


```
def parse_date_safe(val):
    """Parse date strings safely; return NaT on failure."""
    if pd.isna(val) or val == "":
        return pd.NaT
    try:
        return parser.parse(str(val), dayfirst=False, yearfirst=False)
    except Exception:
        return pd.NaT
```

[6]

Python

```
date_cols = ["occ_date", "report_date"]

for col in date_cols:
    if col in df.columns:
        df[col] = df[col].apply(parse_date_safe)

df[date_cols].dtypes
```

[7]

Python

```
occ_date      datetime64[ns]
report_date   datetime64[ns]
dtype: object
```

```
# Quick look at parsed dates
df[date_cols].head()
```

[8]

Python

```
occ_date  report_date
0  2013-12-26  2014-01-01
1  2014-01-01  2014-01-01
2  2014-01-01  2014-01-01
3  2014-01-01  2014-01-02
4  2013-09-30  2014-01-02
```

Converting raw date strings into proper datetime fields

Here I define a helper function `parse_date_safe` that safely converts raw date strings into pandas `datetime` objects and returns `NaT` for any invalid or missing values. I then apply this function to the two key date columns, `occ_date` and `report_date`. The output shows that both columns are now stored as `datetime64` and the preview confirms that the dates are parsed correctly, which is important for doing time-series analysis (by year, month, day, or hour) later in SQL and QuickSight.



```
numeric_cols = [  
    "_id",  
    "occ_year",  
    "occ_day",  
    "occ_doy",  
    "occ_hour",  
    "report_year",  
    "report_day",  
    "report_doy",  
    "report_hour",  
    "bike_speed",  
    "bike_cost",  
    "long_wgs84",  
    "lat_wgs84",  
]  
  
for col in numeric_cols:  
    if col in df.columns:  
        df[col] = pd.to_numeric(df[col], errors="coerce")  
  
df[numeric_cols].dtypes
```

[9]

Python

```
... _id          int64  
    occ_year    int64  
    occ_day     int64  
    occ_doy     int64  
    occ_hour    int64  
    report_year int64  
    report_day  int64  
    report_doy  int64  
    report_hour int64  
    bike_speed  float64  
    bike_cost   float64  
    long_wgs84  float64  
    lat_wgs84   float64  
    dtype: object
```

Converting key fields to numeric types

In this step I define a list of columns that should contain numeric values (years, day-of-week indexes, hours, bike speed/cost, and latitude/longitude). I then loop through this list and use `pd.to_numeric(..., errors="coerce")` to force each column into a numeric type, converting any invalid values to `NaN`. The dtype summary at the bottom confirms that these fields are now stored as `int64` or `float64`, which is required for reliable aggregations, statistics, and mapping in MySQL and QuickSight.

```
# Check a few numeric columns
df[numeric_cols].head()
```

[10]

Python

...

	_id	occ_year	occ_day	occ_doy	occ_hour	report_year	report_day	report_doy	report_hour	bike_speed	bike_cost	long_wgs84	lat_wgs84
0	1	2013	26	360	19	2014	1	1	17	21.0	1300.0	-79.395643	43.640021
1	2	2014	1	1	18	2014	1	1	18	1.0	NaN	-79.414654	43.660525
2	3	2014	1	1	7	2014	1	1	7	10.0	NaN	-79.443645	43.637657
3	4	2014	1	1	12	2014	2	2	20	9.0	1019.0	-79.393760	43.642772
4	5	2013	30	273	0	2014	2	2	12	21.0	750.0	-79.404678	43.648964

▷ ▾

```
text_cols = [
    "event_unique_id",
    "primary_offence",
    "occ_month",
    "occ_dow",
    "report_month",
    "report_dow",
    "division",
    "location_type",
    "premises_type",
    "bike_make",
    "bike_model",
    "bike_type",
    "bike_colour",
    "status",
]

for col in text_cols:
    if col in df.columns:
        df[col] = (
            df[col]
            .astype(str)
            .str.strip()
            .str.upper()
        )

df[text_cols].head()
```

[11]

Python

...

	event_unique_id	primary_offence	occ_month	occ_dow	report_month	report_dow	division	location_type	premises_type	bike_make	bike_model	bike_type	bike_colour	status
0	GO-20141263544	B&E	DECEMBER	THURSDAY	JANUARY	WEDNESDAY	D14	OTHER COMMERCIAL / CORPORATE PLACES (FOR PROF...	COMMERCIAL	FELT	F59	RC	SILRED	STOLEN
1	GO-20141263784	PROPERTY - FOUND	JANUARY	WEDNESDAY	JANUARY	WEDNESDAY	D14	SINGLE HOME, HOUSE (ATTACH GARAGE, COTTAGE, MO...	HOUSE	TREK	SOHO S	RG	BLK	RECOVERED
2	GO-20141261431	THEFT UNDER	JANUARY	WEDNESDAY	JANUARY	WEDNESDAY	D14	APARTMENT (ROOMING HOUSE, CONDO)	APARTMENT	SUPERCYCLE	NAN	MT	NAN	STOLEN
3	GO-20149000090	THEFT UNDER	JANUARY	WEDNESDAY	JANUARY	THURSDAY	D52	APARTMENT (ROOMING HOUSE, CONDO)	APARTMENT	GI	TCX2 (2010)	OT	BLU	STOLEN

Standardizing text columns for consistent analysis

Here I define a list of all categorical/text columns (offence details, dates as words, division, location and premises type, bike make/model/type/colour, and status). For each of these columns I convert the values to string, strip extra spaces, and convert everything to uppercase. The preview at the bottom shows the cleaned text (e.g., months and days in uppercase and without trailing spaces). This makes grouping, filtering, and joining in MySQL and QuickSight much more reliable and avoids issues caused by inconsistent spelling or casing.

```

# Before dropping, check missing values
df[["occ_date", "lat_wgs84", "long_wgs84"]].isna().sum()

[12] Python

...
occ_date      0
lat_wgs84    341
long_wgs84    341
dtype: int64

# Drop rows without occurrence date
if "occ_date" in df.columns:
    df = df.dropna(subset=["occ_date"])

# Drop rows missing coordinates
for coord_col in ["lat_wgs84", "long_wgs84"]:
    if coord_col in df.columns:
        df = df.dropna(subset=[coord_col])

print("Shape after dropping missing occ_date / coordinates:", df.shape)

[13] Python

... Shape after dropping missing occ_date / coordinates: (39132, 30)

D ▾
if "geometry" in df.columns:
    df = df.drop(columns=["geometry"])

before = df.shape[0]
df = df.drop_duplicates()
after = df.shape[0]

print(f"Rows before drop_duplicates: {before}")
print(f"Rows after drop_duplicates: {after}")
print("Current shape:", df.shape)

[14] Python

... Rows before drop_duplicates: 39132
Rows after drop_duplicates: 39132
Current shape: (39132, 29)

```

Removing rows with missing key fields and duplicates

In this step I first check how many missing values we have in the critical columns `occ_date`, `lat_wgs84`, and `long_wgs84`. Then I drop any rows that do not have an occurrence date or valid latitude/longitude, because those records cannot be used for time-based or map-based analysis. After that I remove the `geometry` column (it stores the same location in a complex JSON format, which we don't need once we have clean lat/long) and call `drop_duplicates()` to ensure that duplicate incidents are not counted twice. The printout at the bottom shows how many rows remain and confirms the final cleaned shape of the dataset.

Challenges & Solutions

THE CHALLENGES



EC2 memory limits



Kafka connectivity



Cross-account
S3 access



Data quality issues



THE SOLUTIONS



Upgraded to t3.xlarge



Configured
advertised.listeners



Added bucket policy



Robust pandas cleaning

The screenshot displays the Amazon S3 console interface. The left-hand navigation pane shows the hierarchy: **Amazon S3** > **Buckets** > **toronto-bicycle-thefts** > **clean/**. The main content area is titled **clean/** and has two tabs: **Objects** (selected) and **Properties**. A **Copy S3 URI** button is in the top right corner. Below the tabs, there's a section for **Objects (1)** with a refresh icon and several action buttons: **Copy S3 URI**, **Copy URL**, **Download**, **Open**, **Delete**, **Actions**, **Create folder**, and **Upload**. A descriptive text explains that objects are fundamental entities in Amazon S3 and provides a link to **Amazon S3 inventory**. Below this is a search bar labeled *Find objects by prefix*. A table lists the objects with columns: **Name**, **Type**, **Last modified**, **Size**, and **Storage class**. One object is listed: **Bicycle_Thefts_cleaned.csv** (Type: csv, Last modified: November 19, 2025, 00:42:07 (UTC-05:00), Size: 9.4 MB, Storage class: Standard). The bottom of the console shows a footer with **CloudShell**, **Feedback**, **Console Mobile App**, and copyright information for Amazon Web Services, Inc. or its affiliates, along with links for **Privacy**, **Terms**, and **Cookie preferences**.

This screenshot shows the **toronto-bicycle-thefts** S3 bucket, specifically the **clean/** prefix that contains the processed file **Bicycle_Thefts_cleaned.csv**. This object is the output of the Kafka consumer ETL job running on EC2. S3 acts as our data lake: the cleaned CSV in this location is shared with teammates (via bucket policy) so they can download it, load it into Amazon RDS MySQL, and use it as the source for SQL analysis and QuickSight dashboards.

Aurora and RDS > Databases > toronto-bicycle-thefts-database

Aurora and RDS

- Dashboard
- Databases
- Query editor
- Performance insights
- Snapshots
- Exports in Amazon S3
- Automated backups
- Reserved instances
- Proxies
- Subnet groups
- Parameter groups
- Option groups
- Custom engine versions
- Zero-ETL integrations
- Events
- Event subscriptions
- Recommendations 2
- Certificate update

toronto-bicycle-thefts-database

Summary

DB identifier toronto-bicycle-thefts-database	Status Available	Role Instance	Engine MySQL Community	Recommendations 2 Informational
CPU 3.18%	Class db.t3.micro	Current activity 0 Connections	Region & AZ us-east-1d	

Connectivity & security | Monitoring | Logs & events | Configuration | Zero-ETL integrations | Maintenance & backups | Data migrations | Tags | Recommendations

Connectivity & security

Endpoint & port Endpoint toronto-bicycle-thefts-database.c0psiiuydmz.us-east-1.rds.amazonaws.com Port 3306	Networking Availability Zone us-east-1d VPC vpc-0e63810c72fa82828 Subnet group default-vpc-0e63810c72fa82828 Subnets subnet-0796e25fe2f6b49c0 subnet-0eadb9c331877d627 subnet-0abf46667d7b6408d subnet-0c5913c3fe83fe9e4 subnet-01135d7967243a679 subnet-0a3898281a516ddb5 Network type IPv4	Security VPC security groups bicycle-thefts-rds-sg (sg-041bb42ddced8cca6) Active Publicly accessible Yes Certificate authority Info rds-ca-rsa2048-g1 Certificate authority date May 25, 2061, 19:34 (UTC-04:00) DB instance certificate expiration date November 19, 2026, 01:51 (UTC-05:00)
---	---	--

CloudShell Feedback Console Mobile App

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

This screenshot shows the **toronto-bicycle-thefts-database** Amazon RDS instance that we use as the relational analytics layer. The engine is **MySQL Community** (db.t3.micro) running in **us-east-1d**, exposed on port **3306** with public access enabled so teammates can connect from MySQL Workbench and QuickSight. The endpoint shown here is used to load the cleaned CSV from S3 and create tables/views, and later used as the data source for building the Toronto bicycle theft dashboards.

Explore

Amazon Quick Suite

Datasets

Search

Home

Favorites

CONNECT APPS AND DATA

Integrations

Extensions

QUICK SIGHT

Analyses

Dashboards

Scenarios

Stories

Topics

Datasets

My folders

Shared folders

QUICK AUTOMATE

Datasets

Prepare data for visualization

Combine one or more data sources into structured tables to use in analyses. [Learn more](#)

Create datasetDismiss



Datasets

Data sources

Create data source

Search data sources by name

Name	Source	Owner	Last Modified	Action
 bicycle_thefts_rds	 RDS	Me	14 days ago	

In this step, I created an Amazon QuickSight account to build the dashboards for our project and added a data source that points directly to our RDS MySQL instance. Because QuickSight is natively integrated with RDS, connecting the database and starting to design charts, maps, and filters was straightforward and required almost no extra setup.

Explore

Amazon Quick Suite

Search

Q

Manage account

Account

Control top-level settings and preferences

Account settings

Manage assets

Amazon Q

Subscriptions

Plans your account has enabled

Manage subscriptions

SPICE capacity

Index capacity

Identity

Control who can log into this account

Manage users

Manage groups

SSO

Security

How your data gets shared

Manage domains

Mobile settings

Manage IP/VPC restrictions

Manage users

Invite new users, manage roles for users accessing the Quick Suite account. [Learn more](#)

Invite users

Manage permissions

Search for a user

Q

Role: All

Activity: All

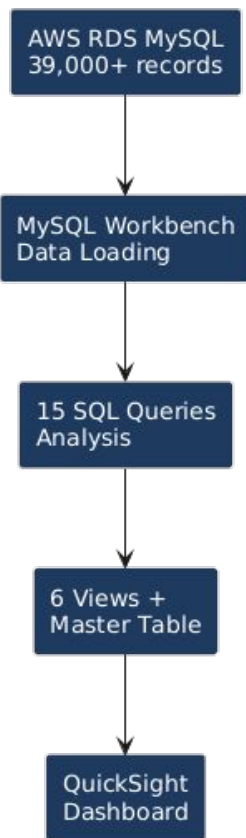
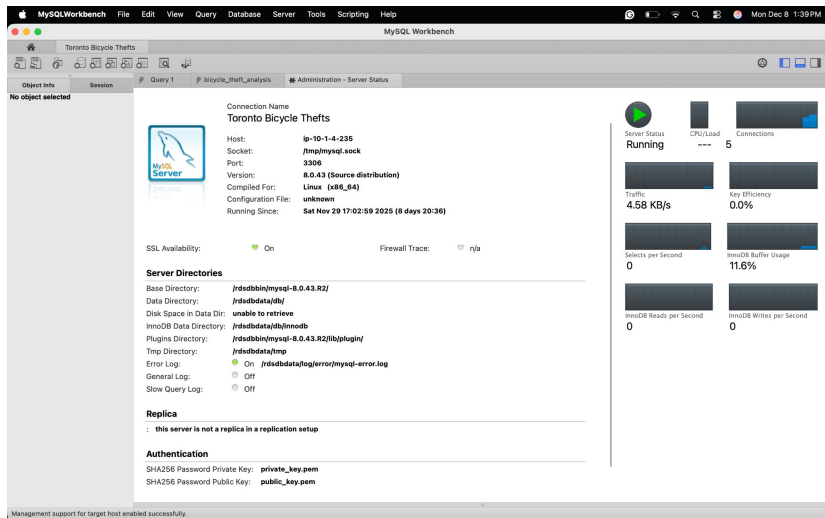
Username	Email	Role	Permissions	Last active	Action
418272788584	chaitanyagandi2000@gmail.com	Admin Pro		2025-12-08 13:30	
han.qi2@northeastern.edu	han.qi2@northeastern.edu	Author		2025-12-08 00:01	Reset password

Showing 1 - 2 of 2 users.

In QuickSight, I also configured access for my team. From **Manage account** → **Manage users**, I invited my teammate using their email and assigned them the **Author** role. This ensures they can log in, connect to the shared RDS data source, and independently build and edit the project dashboards while I keep admin control of the account.

SQL Analysis & Database Design

- Connected to AWS RDS MySQL database
- Loaded 39,000+ bicycle theft records
- Developed 15 analytical queries
- Created 6 views + master table for dashboards



When Do Thefts Happen? - Temporal Analysis

SQL Query: Analyzed theft trends over time

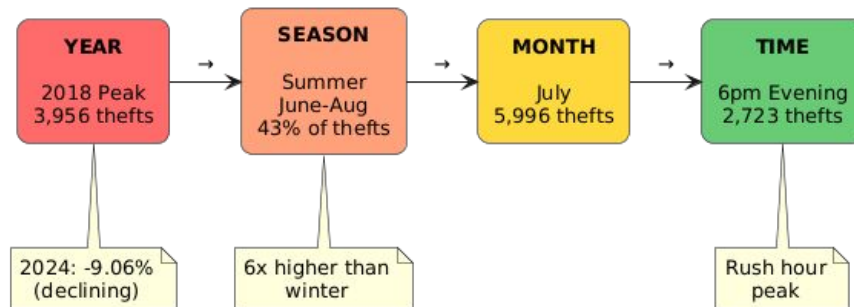
Key Findings:

- **Peak Year:** 2018 with 3,956 thefts
- **2024 Trend:** Declining! Down 9.06% from 2023
- **Peak Month:** July (5,996 thefts - summer cycling season)
- **Peak Hour:** 6pm (2,723 thefts - evening commute)
- **Highest Risk Period:** Evening (6pm-10pm)

Result Grid			Filter Rows: Search
occ_year	total_thefts	percentage	
2014	3031	7.75	
2015	3288	8.40	
2016	3810	9.74	
2017	3872	9.89	
2018	3956	10.11	
2019	3720	9.51	
2020	3886	9.93	
2021	3153	8.06	
2022	2949	7.54	
2023	2981	7.62	
2024	2711	6.93	

hour_of_day	theft_count	time_period
18	2723	Evening (6pm-10pm)
17	2540	Afternoon (12pm-6p...
12	2365	Afternoon (12pm-6p...
19	2282	Evening (6pm-10pm)
0	2204	Night (10pm-6am)
16	2112	Afternoon (12pm-6p...
20	2107	Evening (6pm-10pm)
9	2054	Morning (6am-12pm)
15	2007	Afternoon (12pm-6p...
21	1892	Evening (6pm-10pm)

Theft Risk Timeline - When Thefts Peak



Where Do Thefts Happen? - Geographic Analysis

SQL Query: Analyzed theft locations and recovery rates

Key Findings:

- **Highest Risk:** Outside/Street parking (29.40% of all thefts)
- **Second Highest:** Apartments (24.45%)
- **Recovery Crisis:** ALL locations show <2% recovery rate
- **Best Police Division:** D31 with only 1.64% recovery
- **Worst Division:** D32 with 0.59% recovery
- **Parking Lots:** 9.34% of thefts - also high risk

```
72 location_type,  
73 COUNT(*) as theft_count,  
74 ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM bicycle_thefts WHERE location_type IS NOT NULL), 2) as percentage,  
75 ROUND(AVG(bike_cost), 2) as avg_bike_value  
76 FROM bicycle_thefts  
100% 4775
```

location_type	theft_count	percentage	avg_bike_value
APARTMENT (ROOMING HOUSE, CONDO)	9567	24.45	1102.61
BANK AND OTHER FINANCIAL INSTITUTION...	258	0.66	1058.56
BAR / RESTAURANT	880	2.25	1153.98
COMMERCIAL DWELLING UNIT (HOTEL, MO...	119	0.30	1025.10
COMMUNITY GROUP HOME	8	0.02	519.71
CONSTRUCTION SITE (WAREHOUSE, TRAIL...	44	0.11	1281.84
CONVENIENCE STORES	372	0.95	998.15
DEALERSHIP (CAR, MOTORCYCLE, MARINE...	23	0.06	1811.10
GAS STATION (SELF-FULL, ATTACHED CON...	34	0.09	824.04
GO BUS	9	0.02	774.29
GO STATION	310	0.79	652.74
GO TRAIN	16	0.04	621.80
GROUP HOMES (NON-PROFIT, HALFWAY HO...	14	0.04	1067.40
GROUP HOMES (NON-PROFIT, HALFWAY HO...	36	0.09	876.12
HOMELESS SHELTER / MISSION	57	0.15	1485.55
HOSPITAL / INSTITUTIONS / MEDICAL FACIL...	411	1.05	1024.71
JAILS / DETENTION CENTRES	2	0.01	400.00
NURSING HOME	108	0.28	963.94
OPEN AREAS (LAKES, PARKS, RIVERS)	725	1.85	897.36
OTHER COMMERCIAL / CORPORATE PLAC...	3086	7.89	1201.78
OTHER NON COMMERCIAL / CORPORATE P...	101	1.28	1021.63
OTHER PASSENGER TRAIN	1	0.00	300.00
OTHER PASSENGER TRAIN STATION	8	0.02	706.25
OTHER REGIONAL TRANSIT SYSTEM VEHIC...	3	0.01	775.00
OTHER TRAIN ADMIN OR SUPPORT FACILITY	2	0.01	390.00
OTHER TRAIN TRACKS	1	0.00	228
OTHER TRAIN YARD	2	0.01	1300.00
PARKING LOTS (APT., COMMERCIAL OR NO...	3654	9.34	1141.96
PHARMACY	695	1.78	688.81
POLICE / COURTS (PARKLE BOARD, PHO...	58	0.15	965.27
PRIVATE PROPERTY STRUCTURE (POOL, S...	2632	7.49	1184.30
RECREATION (BOAT, PIER, RECREATION)	67	0.18	1168.26

Toronto Bicycle Thefts - Geographic Risk Levels

#FF6B6B CRITICAL RISK
Outside/Street
29.40% of thefts
Recovery: 1.35%

#FFA07A HIGH RISK
Apartments
24.45% of thefts
Recovery: 0.75%

#FFA07A HIGH RISK
Houses
14.34% of thefts
Recovery: 1.18%

#FFD93D MEDIUM RISK
Commercial
12.31% of thefts
Recovery: 0.87%

#6BCB77 LOW RISK
Transit
2.14% of thefts
Recovery: 0.60%

Risk Level Legend:
■ **CRITICAL RISK** - Over 25% of thefts
■ **HIGH RISK** - 15-25% of thefts
■ **MEDIUM RISK** - 10-15% of thefts
■ **LOW RISK** - Under 5% of thefts

What Gets Stolen? - Bike Brand & Seasonal Analysis

SQL Query: Analyzed most stolen bike brands and seasonal patterns

Key Findings:

Bike Brands:

- **Most stolen:** OT brand (7,879 thefts, avg value \$1,021)
- **Second:** UK brand (4,209 thefts, avg value \$887)
- **Premium brands:** Specialized (\$1,880 avg) shows 2.97% recovery
- **945 bikes** with "UNKNOWN" make - highlights registration need

Seasonal Patterns:

- **Summer (June-Aug):** 16,894 thefts (43% of annual)
- **Winter (Dec-Feb):** Only 3,229 thefts
- **6x higher risk** in summer vs winter
- **Winter bikes:** Higher average value (\$1,300-\$1,400)

```
97 -- 8. Most stolen bike makes (brands thieves target)
98 * SELECT
99     bike_make,
100     COUNT(*) as times_stolen,
101     ROUND(AVG(bike_cost), 2) as avg_value,
102     COUNT(CASE WHEN status = 'RECOVERED' THEN 1 END) as recovered,
103     ROUND(COUNT(CASE WHEN status = 'RECOVERED' THEN 1 END) * 100.0 / COUNT(*), 2) as recovery_rate
104 FROM bicycle_thefts
105 WHERE bike_make IS NOT NULL
106 AND bike_make != ''
107 AND bike_make != 'UNKNOWN'
```

bike_make	times_stolen	avg_value	recovered	recovery_rate
OT	7879	1021.76	51	0.65
UK	4209	887.57	11	0.26
GI	2267	982.58	14	0.62
OTHER	2172	1482.28	54	1.57
TR	2044	1008.26	8	0.39
NO	1275	822.87	6	0.47
UNKNOWN MAKE	945	966.70	12	1.27
CC	859	419.10	1	0.12
TREK	844	1545.46	16	1.90
SU	832	270.73	0	0.00
GIANT	834	1261.60	19	2.36
CA	782	1397.46	4	0.51
RA	728	579.93	6	0.82
SPECIALIZED	708	1880.00	21	2.97
SC	705	498.78	0	0.00

Top 5 Stolen Brands

#1 OT
7,879 thefts
\$1,021 avg
0.65% recovery

#2 UK
4,209 thefts
\$887 avg
0.26% recovery

#3 Giant
2,267 thefts
\$982 avg
0.62% recovery

#4 Trek
2,044 thefts
\$1,008 avg
0.39% recovery

#5 Specialized
708 thefts
\$1,880 avg
2.97% recovery

945 bikes "UNKNOWN"
Registration critical!

Data Products for Dashboards

SQL Views & Master Table

SQL Views Created for QuickSight:

I built **6 analytical views** to enable interactive dashboards:

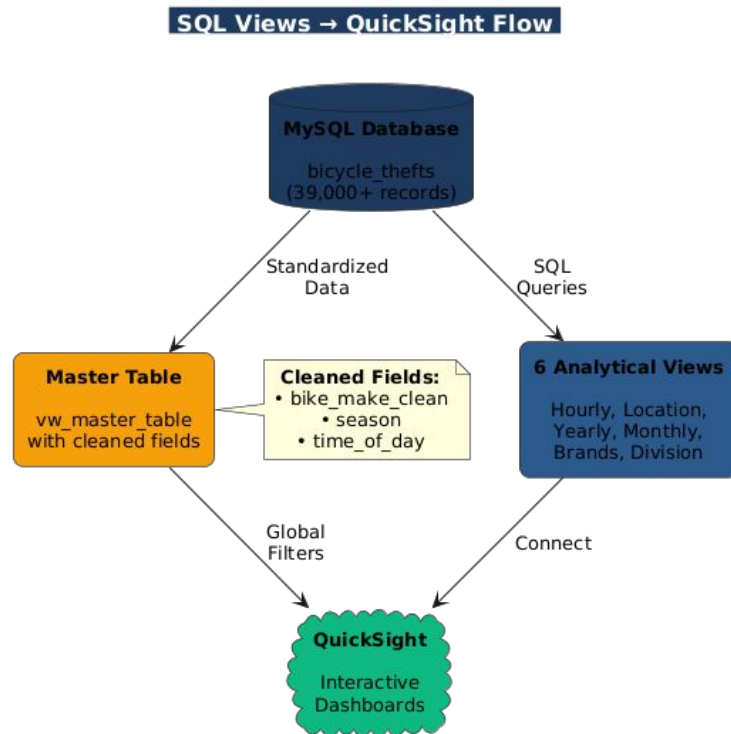
- **vw_hourly_thefts** - Time pattern analysis
- **vw_top_locations** - Geographic hotspots with recovery rates
- **vw_yearly_trends** - Year-over-year theft trends
- **vw_monthly_patterns** - Seasonal analysis with bike values
- **vw_bike_brands** - Brand targeting with recovery metrics
- **vw_division_performance** - Police division performance

Master Table (vw_master_table):

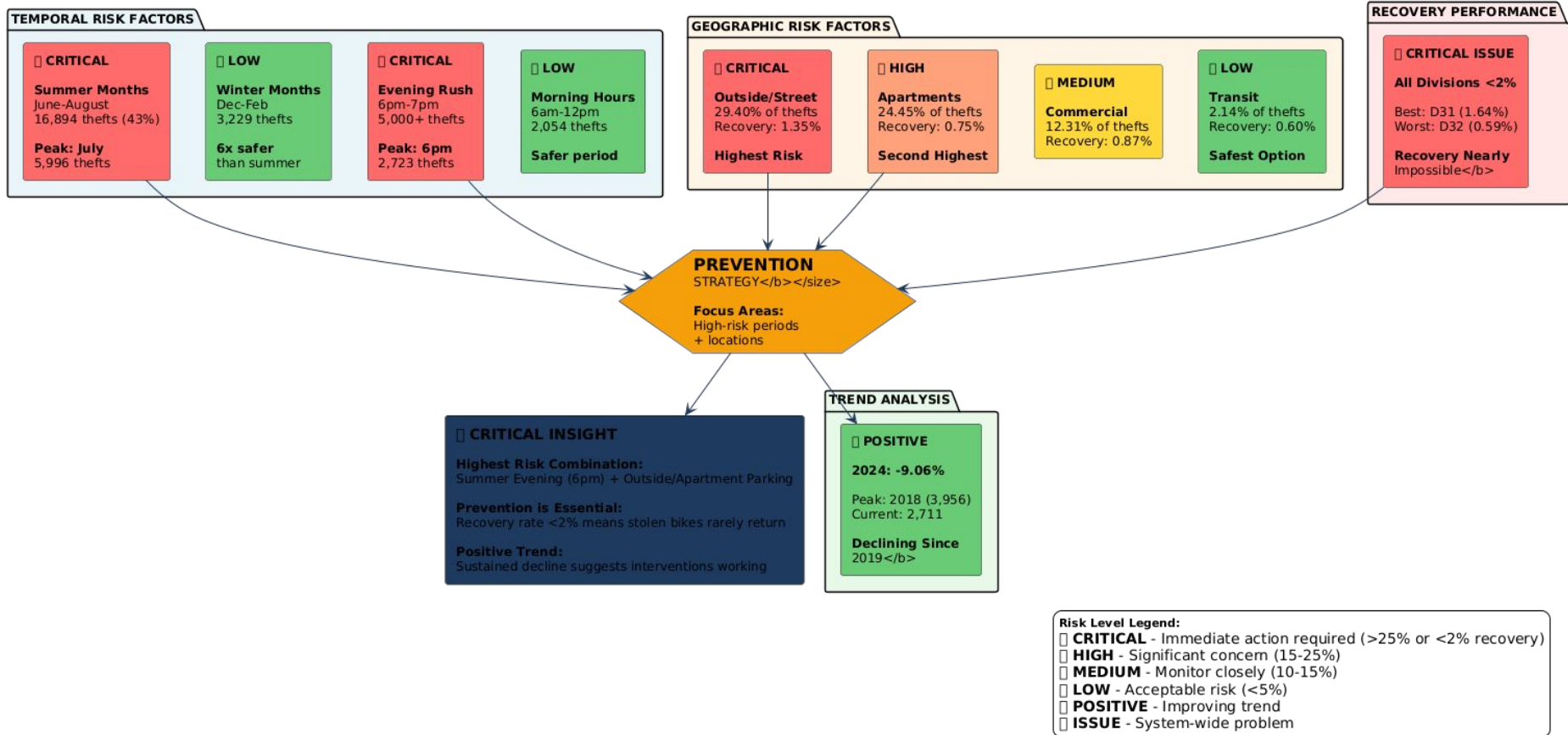
Built for interactive filtering and global dashboards:

- **bike_make_clean** - Standardized brand names (GIANT, TREK, etc.)
- **season** - Winter/Spring/Summer/Fall classification
- **time_of_day** - Morning/Afternoon/Evening/Night periods

Purpose: Enable Qi to create dynamic dashboards with filters



Toronto Bicycle Thefts - Comprehensive Risk Assessment Matrix



Recommendations & Action Plan

4-Pillar Prevention Strategy

PILLAR 1 **CYCLISTS** Personal Defense

Key Actions:

- Quality locks (2+ types)
- Avoid summer evenings
- Register serial numbers
- Indoor parking priority
- Bike insurance coverage

PILLAR 2 **POLICE** Smart Deployment

Strategic Focus:

- Summer patrol increase
- Evening presence (5-7pm)
- Apartment/outdoor focus
- Bike theft task force
- Online marketplace watch

PILLAR 4 **PROPERTY** Infrastructure

Building Security:

- Indoor bike storage
- Security upgrades
- Surveillance cameras
- Adequate lighting
- Tenant support

PILLAR 3 **CITY POLICY** System Solutions

Long-term Programs:

- Mandatory registration
- GPS tracker programs
- Secure parking hubs
- Public awareness campaigns
- Tech partnerships

Implementation Priority:

Phase 1 (Immediate): Cyclist education + Police summer deployment

Phase 2 (3-6 months): Property upgrades + Registration pilot

Phase 3 (6-12 months): City-wide programs + Tech integration

Target: 25% reduction in thefts within 12 months

Why This Strategy Works:

Data-Driven Approach:

- Analysis identified clear patterns in when, where, and what gets stolen
- Each pillar targets a specific high-risk factor from our findings

Multi-Stakeholder Solution:

- No single group can solve alone - requires coordinated effort
- Addresses all risk dimensions: temporal, geographic, recovery

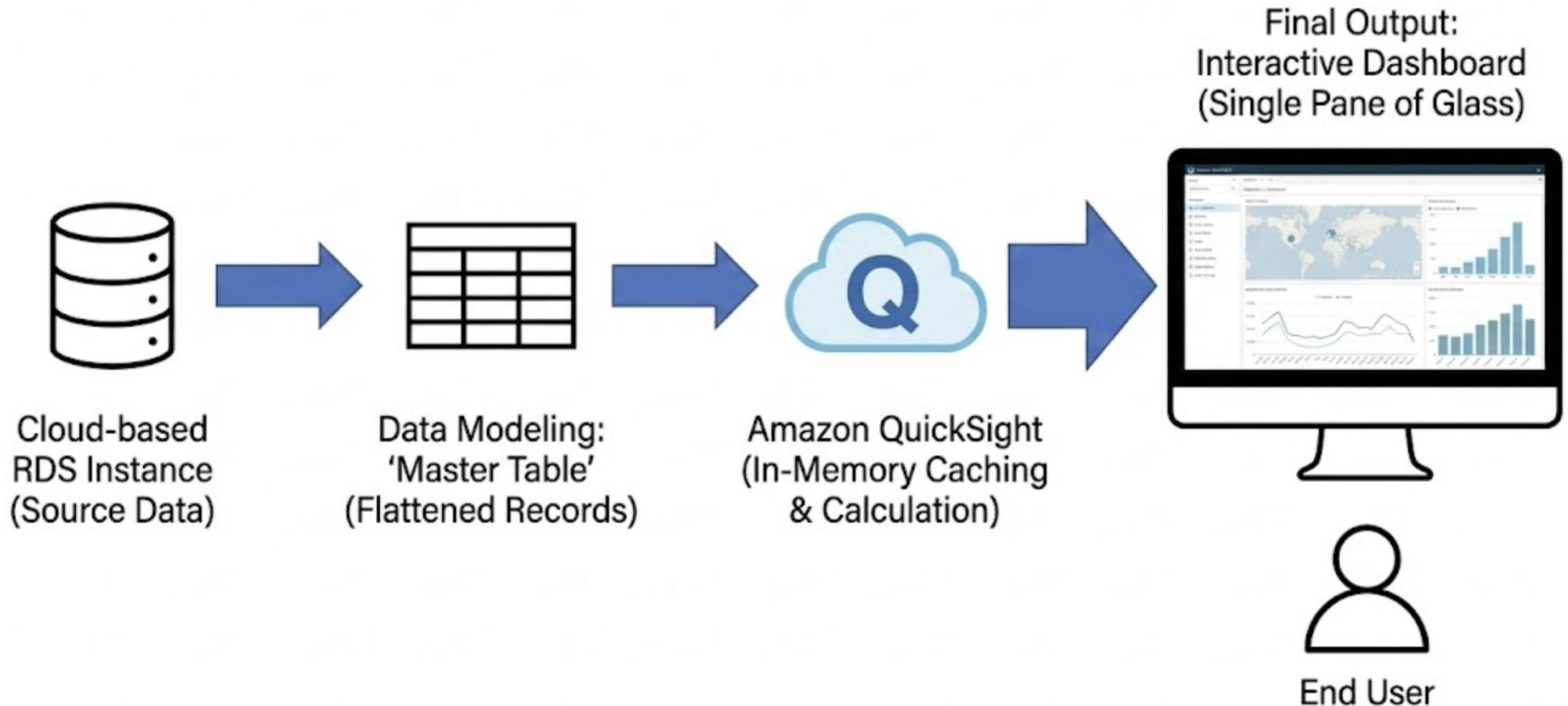
Proven Success Potential:

- 2024 shows 9.06% decline - interventions are working
- Strategic focus can accelerate this positive trend

Prevention Over Recovery:

- Current recovery rate: <2% across all divisions
- Prevention is the only viable strategy
- Coordinated approach maximizes impact

BI Implementation & Visual Analytics



Technical Architecture & Backend Strategy

BI Platform Selection: Amazon QuickSight (Cloud-native integration with RDS).

Data Modeling Strategy:

- Moved from fragmented SQL views to a single **"Master Table" architecture**.
- Enables dynamic aggregation and flexible filtering on the fly.

Performance Optimization (SPICE):

- Utilized **S**uper-fast, **P**arallel, **I**n-memory **C**alculation **E**ngine.
- **Result:** Sub-second load times for 10+ years of geospatial data.

Visual Analysis – The "Where" and "When"

Spatial Analysis (The Map):

- **Visualization Type:** Interactive **Point Map** (Pin-level Granularity).
- **Implementation:** Plotted individual theft coordinates (Latitude/Longitude) to visualize exact incident locations rather than aggregated regions.
- **Insight Logic:** Applied a conditional color schema (**Orange = Stolen**, **Blue = Recovered**), allowing users to identify precise "recovery hotspots" amidst the high-density theft zones.



Visual Analysis – The "Where" and "When"

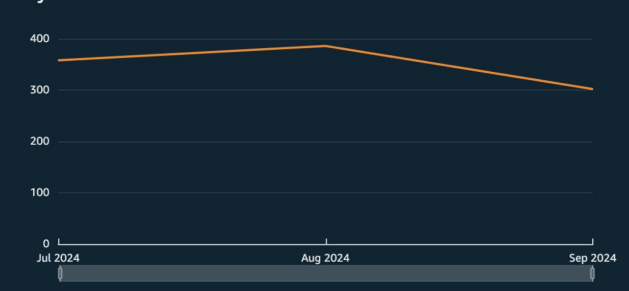
Temporal Analysis (The Trend):

- **Analysis Type:** 10-Year Time Series tracking theft volume.
- **Interactive Feature:** Implemented a **Drill-Down Hierarchy (Year/Quarter/Month)**.
- **Why it matters:** Allows users to switch between viewing long-term annual trends and investigating specific seasonal "waves" (Summer spikes) without changing the chart view.

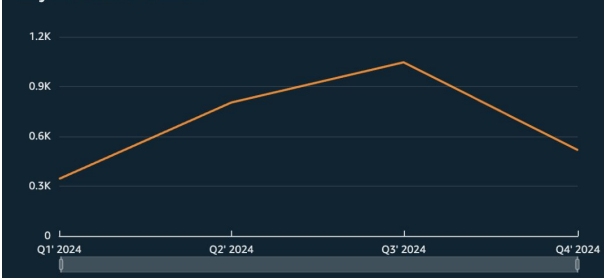
Bicycle Theft Trends



Bicycle Theft Trends

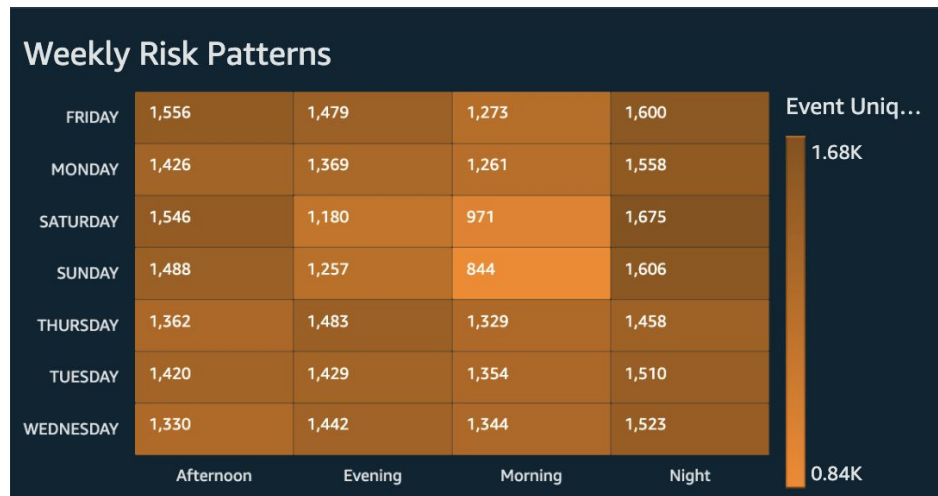


Bicycle Theft Trends



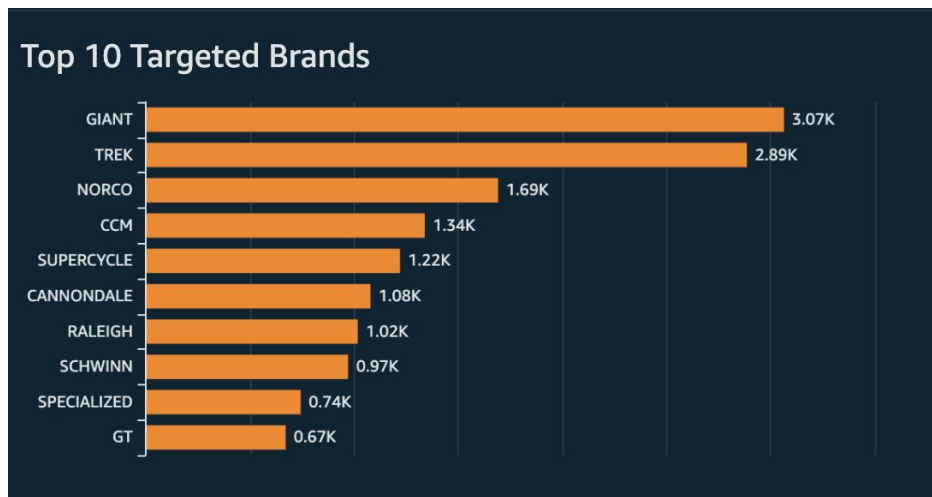
Behavioral Risk & Target Profiling

- **Behavioral Risk (The Heatmap):**
 - **Method:** Matrix crossing *Day of Week* vs. *Time of Day*.
 - **Discovery:** Isolates "Danger Zones" (e.g., Friday Evenings) that simple line charts miss.



Behavioral Risk & Target Profiling

- **Target Analysis (The Bar Chart):**
 - **Method:** Top 10 Ranking Filter on **Bike_Make_Clean**.
 - **Constraint:** Filtered "Unknown" data to focus on actionable insights.
 - **Insight:** Identifies premium brands (Giant, Trek) as primary targets for organized theft.



Dashboard Design & User Experience

"Single Pane of Glass" Architecture:

- Implemented **Free-form layout** with auto-scaling ("Fit to Window"), ensuring the entire dashboard is visible without scrolling on any device.

Global Filter Suite:

- Five interconnected controls (**Year, Neighborhood, Bike Type, Day, Time**) allow users to slice the entire dataset instantly.

Omni-Directional Interactivity:

- Unified Action Layer:** Interactivity is enabled on **all components**, including the **4 KPI Cards**, Map, Trend Line, Heatmap, and Bar Chart.
- Context-Aware Metrics:** Clicking any visual (e.g., a map cluster) filters all other charts and **instantly recalculates KPI totals** to reflect the selected data.



Conclusion & Critical Insights

1. The Good News: Declining Trend

- Bicycle thefts dropped over **9%** in 2024, continuing a multi-year downward trajectory.

2. The Bad News: Recovery Crisis

- The recovery rate remains extremely low at **less than 2%** across all police divisions.

3. High-Risk Locations

- **Apartments** are the most vulnerable premises type, accounting for nearly **25%** of all reported thefts.

4. "Danger Zones" (Time & Season)

- Risk is not random. **Summer months** see **6x more thefts** than winter, peaking specifically on July evenings.

5. Targeted Assets

- Thieves are brand-aware. Premium brands like **Giant and Trek** are primary targets, while "Unknown" bikes still account for significant losses.

THANK YOU